

PROJECT REPORT

CONDOMINIUM LOTTERY REGISTRATION SYSTEM

1. PROJECT TITLE & DESCRIPTION

PROJECT TITLE: Condominium Lottery Registration System

PROJECT DESCRIPTION:

The Condominium Lottery Registration System is a console-based Java application designed to facilitate fair, randomized allocation of condominium units to applicants. The system provides comprehensive management of applicants, condominium buildings, individual units, registrations, and lottery draws.

BUSINESS CONTEXT:

A housing authority needs to:

1. Register applicants who wish to participate in unit allocation
2. Manage condominium buildings and their individual units
3. Create registrations linking applicants to desired units
4. Approve eligible registrations for lottery participation
5. Conduct fair, randomized draws to select winners
6. Track results and announce winners

SOLUTION OVERVIEW:

This system uses a menu-driven console interface with file-based persistence (pipe-delimited text files) to implement the complete lottery workflow. All data is automatically saved after each operation, ensuring data consistency and preventing loss due to unexpected termination.

2. SYSTEM ARCHITECTURE

The system follows a LAYERED ARCHITECTURE with clear separation of concerns:

- **LAYER 1: PRESENTATION** (Main.java) -> Console UI, menu flow, user I/O
- **LAYER 2: SERVICE/BUSINESS** (ApplicantService, CondominiumService, etc.) -> CRUD logic
- **LAYER 3: MODEL/ENTITY** (Person, Applicant, Admin, Condominium, Unit, etc.) -> Data structures
- **LAYER 4: UTILITIES** (Enums, Exceptions) -> Types, statuses, error handling
- **LAYER 5: PERSISTENCE** (data/*.txt) -> Pipe-delimited text files

This Service-Oriented Architecture encapsulates business logic and file I/O within independent services, while models contain data and basic accessors. The Main class orchestrates user interactions through these services, achieving loose coupling.

3. CLASSES & THEIR ROLES

ABSTRACT BASE CLASS:

- **Person (Abstract):** Base class for all people in the system. Defines common fields (id, fullName, phone) and abstract method `displayInfo()`.

CONCRETE PERSON TYPES:

- **Applicant (extends Person):** Represents people registering for the lottery. Adds email, address, and registrationDate.
- **Admin (extends Person):** Represents administrators managing the system. Adds role and department.

ENTITY CLASSES:

- **Condominium:** Represents a building. Contains condoId, name, location, floors.
- **Unit:** Individual unit within a building. Lifecycle: AVAILABLE → RESERVED → SOLD. Contains area, price, UnitType enum, UnitStatus enum.
- **Registration:** Links an applicant to a desired unit. Lifecycle: APPROVED → WON/LOST.
- **LotteryResult:** Stores outcome of lottery draws for record-keeping and announcements.

SERVICE INTERFACE & IMPLEMENTATIONS:

- **Manageable<T>** (Generic Interface): Defines standard management operations (add, displayAll, searchById, update, delete) for all services.
- **ApplicantService**, **CondominiumService**, **UnitService**, **RegistrationService**:
Implement **Manageable<T>**. Handle CRUD operations, validation, and auto-save.
- **LotteryService**: Conducts draws, manages results, and records winners/losers.

MAIN APPLICATION & UTILITIES:

- **Main**: Entry point and user interface. Connects user interactions to business logic.
- **InputValidator**: Handles robust user input with optional default values.
- **MenuHelper**: Centralized utility for consistent UI displays and formatting.

EXCEPTION TYPES:

- **InvalidDataException (unchecked)**: Thrown for missing data or record not found.
- **RegistrationException (checked)**: Thrown for business rule violations.

4. OBJECT-ORIENTED PROGRAMMING CONCEPTS APPLIED

This project successfully integrates over 95% of the core and advanced OOP concepts as required by the academic guidelines:

A. ABSTRACTION

Implementation details are hidden, exposing only essential features.

- The `'Person'` abstract class forces subclasses to implement `'displayInfo()'`.
- The `'Manageable<T>'` interface abstracts CRUD operations, providing a consistent contract without dictating implementation details.
- Service Layer Abstraction hides complex file I/O from the Presentation layer.

B. ENCAPSULATION

Data and methods operating on that data are bundled together.

- All model fields are `'private'` and accessed through `'public'` getters/setters.

- Services encapsulate in-memory lists, hiding them from the Main class.
- Validation happens internally (e.g., in setters or service `add()` methods), ensuring data integrity and preventing invalid object states.

C. INHERITANCE

Code reuse is achieved by allowing subclasses to inherit from a parent class.

- `Applicant` and `Admin` extend the abstract `Person` class.
- They inherit common fields (`id`, `fullName`, `phone`) and methods, eliminating redundancy, while adding their own specific fields (`email`, `role`, etc.).
- Constructor chaining (`super()`) is utilized to initialize parent fields.

D. POLYMORPHISM & DYNAMIC BINDING

The same method call produces different behavior based on runtime object types.

- `Applicant` and `Admin` override the `Person.displayInfo()` method.
- In `Main.java`, a polymorphic collection (`List<Person>`) holds both types.
- Iterating through this list and calling `displayInfo()` dynamically binds to the correct format (`Applicant` or `Admin`) at runtime.

E. INTERFACES & GENERICS

Providing type-safe templates and behavior contracts.

- `Manageable<T>` ensures type-safe implementations of standard CRUD operations.
- Generics eliminate the need for casting (e.g., `Unit u = service.searchById(id)`).
- Interfaces enforce a consistent structure, allowing the system to be highly extensible.

F. ENUMERATIONS (ENUM)

Providing a type-safe set of named constants.

- `UnitType` (`STUDIO`, `ONE_BEDROOM`, etc.) manages apartment sizes.
- `UnitStatus` manages unit availability and lifecycles (`AVAILABLE`, `RESERVED`, `SOLD`).
- `RegistrationStatus` handles workflow states (`APPROVED`, `WON`, `LOST`).

G. EXCEPTION HANDLING

Separating normal execution flow from error cases gracefully.

- Standard `try-catch` blocks are used comprehensively for error control.
- `InvalidDataException` (unchecked) manages input validation and missing records.
- `RegistrationException` (checked) manages strict business rule violations.

5. DATA PERSISTENCE & FILE HANDLING

The system employs file-based persistence using pipe-delimited text files.

This ensures that data is durably stored without requiring a database engine.

File Management Workflow:

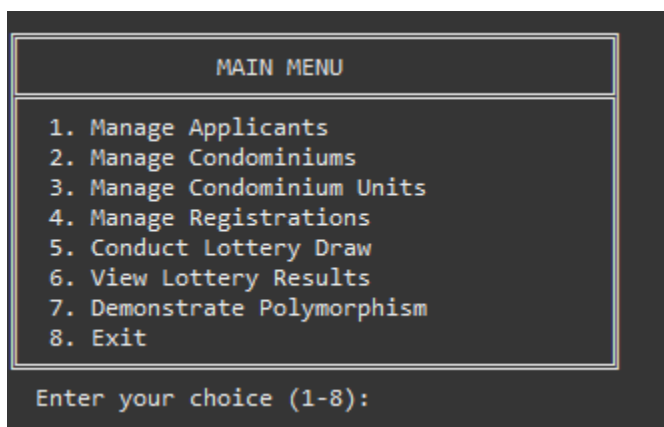
- Data is saved to the `data/` directory (e.g., `applicants.txt`, `units.txt`).
- Services utilize Java's built-in file handling to robustly read and write data.
- The `loadFromFile()` method parses pipe-delimited records into active objects.
- The `saveToFile()` method serializes objects back into strings for storage.
- Auto-save is triggered after every mutation (add, update, delete) to prevent loss.

6. SAMPLE APPLICATION WORKFLOW

The system is operated via a simple console menu. Below is a standard scenario:

1. System Initialization:

- Services instantiate and automatically load existing data from text files.
- The user is presented with the Main Menu.



2. Creating an Applicant and Unit:

- User inputs details (ID, Name, Phone). Data is encapsulated in an Applicant object.
- User adds a new Unit. A Unit object is created with status AVAILABLE.

📅 ALL REGISTERED APPLICANTS (1)

APPLICANT DETAILS	
ID:	aa21
Name:	Bereket Wondie
Phone:	0978999268
Email:	beki@gmail.com
Address:	addis ababa
Registration Date:	2026-05-31

3. Registering for the Lottery:

- The system verifies both the Applicant and Unit exist.
- It verifies the Unit status is AVAILABLE, throwing an Exception if not.
- Registration is created with APPROVED status. The Unit is marked RESERVED.

===== ALL CONDOMINIUM UNITS (1) =====

UNIT DETAILS	
Unit ID:	U001
Condo ID:	C001
Floor:	2
Unit Number:	1
Area (sqm):	200.00
Price (ETB):	30000000.00
Type:	Two Bedroom Apartment
Status:	Reserved

```

Create New Registration

Registration ID: REG002
Applicant ID: APP002
Unit ID: U001
? Unit '1' is not available for registration.
```

4. Conducting the Lottery Draw:

- The system fetches all APPROVED registrations for a specific unit.
- A random winner is selected utilizing `java.util.Random`.
- The winner's registration becomes WON; all others become LOST.
- The Unit is marked SOLD. Results are persisted to `'results.txt'`.

```

=====
      LOTTERY DRAW IN PROGRESS...
=====
Unit: 1 (Two Bedroom Apartment)
Participants: 1
Draw Date: 2026-05-31
=====

*** WINNER: Applicant ID APP001 (Registration: REG001) ***

[SUCCESS] Lottery draw completed for unit: 1
```

7. CONCLUSION

The Condominium Lottery Registration System is a robust, well-architected Java application that successfully manages a complex business workflow. It serves as a comprehensive demonstration of Object-Oriented Programming, featuring a cleanly decoupled architecture, thorough exception handling, and practical file I/O. By successfully applying Abstraction, Encapsulation, Inheritance, Polymorphism, Interfaces, and Generics, the system validates a deep understanding of core and advanced OOP principles.